Lab 6: 3D Reconstruction with the TurtleBot3*

EECS/ME/BIOE C106A/206A Fall 2026

Goals

Over the course of this lab you will:

- Calibrate a camera with a checkerboard and explain what each intrinsic parameter means
- Collect images of two objects, along with the robot's pose from SLAM, and reconstruct them together
- Detect and match SIFT features, and explain why the ratio test is needed
- Estimate the fundamental matrix, draw epipolar lines, and explain the epipolar constraint
- Recover the relative camera pose from the essential matrix and triangulate a sparse point cloud
- Explain why a single camera cannot measure scale, and use the robot's odometry to recover it
- Refine a reconstruction with bundle adjustment and interpret the result

---

Helpful Documentation:

- `OpenCV camera calibration`: `https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html`
- `OpenCV SIFT`: `https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html`
- `OpenCV feature matching`: `https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html`
- `OpenCV epipolar geometry`: `https://docs.opencv.org/4.x/da/de9/tutorial_py_epipolar_geometry.html`
- `OpenCV calib3d` (`findFundamentalMat`, `recoverPose`, `triangulatePoints`): `https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html`
- Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, Chapters 9–12

Contents

0 Initial Setup **2**

1 Camera Calibration **3**
1.1 The pinhole model and lens distortion 3
1.2 Why a checkerboard 4
1.3 Capturing calibration images 4
1.4 Writing `calibrate.py` 4

2 Capturing a Scan **5**
2.1 Driving and capturing 5

*Written Fall 2026.

3 Features and Matching **6**
3.1 How SIFT works . . . 6
3.2 Matching and the ratio test . . . 6
3.3 Writing `features.py` . . . 6

4 Epipolar Geometry **7**
4.1 The epipolar constraint . . . 7
4.2 How `cv2.findFundamentalMat` works . . . 7
4.3 Choosing a pair . . . 8
4.4 Writing the epipolar part of `two_view.py` . . . 8

5 Two-View Reconstruction **8**
5.1 From F to the essential matrix . . . 8
5.2 Four solutions and the cheirality check . . . 9
5.3 Triangulation . . . 9
5.4 Scale, and where SLAM comes in . . . 9
5.5 Writing the rest of `two_view.py` . . . 9

6 Multi-View Reconstruction **9**

7 Bundle Adjustment **10**

8 Before you Leave **10**

0 Initial Setup

As usual, every step of the setup is required. Please follow carefully. Enter the Distrobox and pull the Lab 6 starter code from the `starter` remote, the same way you did in Lab 4:

```
cd ~/ros_workspaces
git fetch starter
git restore --source=starter/main --staged --worktree -- lab6
```

If you haven't already done so in Lab 5, we will make a `.gitignore` file in our repository, which tells Git which files and directories should not be tracked or committed to the repository. Create a file titled `.gitignore` in the `~/ros_workspaces` folder and copy the following into it:

```
# ROS 2 / colcon
**/build/
**/install/
**/log/
.colcon/

# Python
**/__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/

# IDEs / editors
.vscode/
.idea/

# OS files
.DS_Store
Thumbs.db

# Python virtual environments
```

```
.venv/
venv/
env/
```

The reason we especially need this is because the `build`, `install`, and `log` directories contain automatically generated files that should never be committed. If you accidentally commit them, they can create **merge conflicts later**.

Next run:

```
git rm -r --cached --ignore-unmatch -- '**/build/**' '**/install/**' '**/log/**'
git add .gitignore
git commit -m "Add Lab 6 starter code"
git push origin main
```

You now have:

- `lab6/src/lab6_capture`: a ROS package with two nodes, `hold_teleop` (drive while a key is held) and `capture_pose` (save a camera image and the robot's pose when you press space). These are given.
- `lab6/reconstruction`: the Python scripts you will complete. They run on the workstation, not on the robot, and do not need ROS.

Build the capture package:

```
cd ~/ros_workspaces/lab6
colcon build
source install/setup.bash
```

Python environment for the reconstruction

The reconstruction uses NumPy, SciPy and OpenCV, which are already in the container, and `viser` for 3D viewing, which is not. Make a virtual environment that reuses the container's packages and adds `viser`:

```
cd ~/ros_workspaces/lab6
python3 -m venv --system-site-packages .venv
.venv/bin/pip install viser
```

In every terminal where you run the reconstruction scripts, activate it first:

```
source ~/ros_workspaces/lab6/.venv/bin/activate
cd ~/ros_workspaces/lab6/reconstruction
```

1 Camera Calibration

Everything in this lab turns pixels into rays in 3D. To do that you need to know how the camera maps 3D points to pixels. That mapping is the camera's *intrinsics*, and finding it is called calibration.

1.1 The pinhole model and lens distortion

A point (X, Y, Z) in the camera's frame (z pointing forward, x right, y down) lands on pixel

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \qquad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

where $\sim$ means "equal up to scale" (divide by the third coordinate). f_x and f_y are the focal length in pixels, and (c_x, c_y) is the principal point, where the optical axis meets the image.

Real lenses also bend straight lines, most visibly near the edges of the image. OpenCV's model (`plumb_bob` in ROS) describes this with five numbers $(k_1, k_2, p_1, p_2, k_3)$: k_1, k_2, k_3 for radial distortion (barrel or pincushion) and p_1, p_2 for a lens that is slightly tilted relative to the sensor.

The robot publishes a `camera_info` topic that claims $f_x = f_y = 700$, $c_x = 320$, $c_y = 240$. Those round numbers are placeholders, not a calibration. You will measure the real values.

1.2 Why a checkerboard

A checkerboard gives you many points whose 3D positions you know exactly: its inner corners lie on a flat grid with known spacing. Each photo of the board gives a homography between the board's plane and the image, and each homography gives two equations on K. With enough photos taken from different angles, K can be solved in closed form. OpenCV then refines K, the distortion and every board pose together by minimizing the *reprojection error*: the pixel distance between each detected corner and where the model predicts it (Zhang's method).

Different angles matter: a board seen straight-on, in the same place every time, gives nearly the same equations over and over.

1.3 Capturing calibration images

Get a printed checkerboard from your TA.

Turn on your TurtleBot and start its bringup and camera as in Lab 4 (two ssh sessions):

```
ros2 launch turtlebot3_bringup robot.launch.py
ros2 launch usb_cam usb_cam.launch.py
```

On the workstation, check the camera's topic name with `ros2 topic list | grep image_raw`. Some robots publish `/image_raw`; others publish `/ROBOTNAME/image_raw`. Then start the capture node in images-only mode (no SLAM needed):

```
ros2 run lab6_capture capture_pose --ros-args \
    -p with_pose:=false -p image_topic:=/ROBOTNAME/image_raw
```

Hold the board in front of the camera and press space to save a picture. It goes into:

```
~/ros_workspaces/lab6/captures/calib_<date>_<time>/
```

Take 15–25 pictures:

- Tilt the board forward, backward and sideways, by up to about 45 degrees.
- Put it near the center, in each corner of the image, and along the edges, where distortion is strongest.
- Vary the distance: some where the board fills most of the image, some where it is smaller.
- The whole board must be visible, and it must be still: hold it steady, then press space.

1.4 Writing `calibrate.py`

Task 1: Complete `calibrate.py`:

- **TODO 1.1** `board_points`: the 3D corner positions of the board, in meters, on the plane $z = 0$, in the order OpenCV reports corners (row by row, left to right).
- **TODO 1.2** `find_corners`: call `cv2.findChessboardCorners`, and if the board was found, refine the corners with `cv2.cornerSubPix`.
- **TODO 1.3** `calibrate`: call `cv2.calibrateCamera`.

`--cols` and `--rows` count *inner* corners, where four squares meet: a board with 8×7 squares has 7×6 inner corners. With 25 mm squares:

```
python3 calibrate.py ../captures/calib_<date>_<time> --cols 7 --rows 6 --square 0.025
```

It prints K, the distortion coefficients, the overall RMS reprojection error, and the error for each image, and writes `calibration/calibration.json`. Look at a few images in `calibration/corners/` to check the corners were found in the right places.

An RMS error under about 0.5 px is good. If one image has a much larger error than the others, it is probably blurry or the board moved: delete it and run again.

Checkpoint 1

Submit a checkoff request at `https://tinyurl.com/ee106a-fa26lab`. At this point you should be able to:

- Show your calibration output, including the overall RMS reprojection error
- Show a few of your corner images.
- Explain what f_x, f_y, c_x and c_y mean physically, and how your f_x compares with the 700 in `camera_info`
- Why do we have to undistort the image/How do we model image distortion?
- Explain why you need pictures with the board at different angles. What would go wrong if every picture had the board straight-on in the center?
- Explain what the reprojection error measures

---

2 Capturing a Scan

Now collect the pictures you will reconstruct: 20–30 photos of two objects, each with the robot's pose. Make sure you only move/translate 4-5 cm per image so you can have higher feature matches, this will lead to a better 3D reconstruction

Get two objects from your TA. Place them on the floor next to each other, in an open area at least a meter from walls. You will reconstruct **both objects together in one scan**, so both must appear in every picture.

2.1 Driving and capturing

With bringup and the camera still running, start SLAM on the workstation with the launch file you wrote in Lab 4. It brings up Cartographer and RViz, so you can watch the map build as you drive:

```
source ~/ros_workspaces/lab4/install/setup.bash
ros2 launch turtlebot_controller slam_launch.py
```

In a second terminal, start the capture node, this time recording the robot's pose:

```
source ~/ros_workspaces/lab6/install/setup.bash
ros2 run lab6_capture capture_pose --ros-args -p image_topic:=/ROBOTNAME/image_raw
```

In a third terminal, start the teleop node:

```
source ~/ros_workspaces/lab6/install/setup.bash
ros2 run lab6_capture hold_teleop
```

`hold_teleop` drives while a key is held and stops when you let go: `w`/`x` forward and back, `a`/`d` turn in place, `q`/`e`/`z`/`c` drive while turning.

Each time you press space, `capture_pose` saves the image and the transform `map` → `base_link` at the moment the image was taken, into `captures/run_<date>_<time>/` (images plus `poses.csv`).

Drive a full circle around both objects, about 0.4–0.6 m from them, and capture as you go:

- **Move, stop, then capture.** Let go of the key, wait a second for the robot to settle, then press space.
- **Keep both objects in the image, centered between them.**
- **Move 10–20 cm or turn about 15 degrees between pictures.** Neighboring pictures must overlap a lot. But the camera has to actually *move*: two pictures taken while spinning in place have no parallax, and you cannot triangulate from them.
- **Finish where you started**, so the last picture overlaps the first.
- Avoid people walking through the shot: a moving person is a moving object.

Where is the camera? The pose you record is `base_link`'s, the center of the robot. The reconstruction needs the camera's pose, so `common.py` includes the camera's position on the robot, `T_BASE_CAM`. We measured the lab's TurtleBots and found the camera points about 8 degrees *up*, not straight ahead; that tilt is included. (Lab 4's `tag_launch.py` assumes a level camera.)

3 Features and Matching

To reconstruct a point, you have to find the same point in two or more images. SIFT finds distinctive points and describes each one so that it can be recognized in another image.

3.1 How SIFT works

- **Detection.** SIFT blurs the image at a range of scales and subtracts neighboring blur levels (the *difference of Gaussians*). Blobs and corners give strong extrema in this stack. Each extremum is a keypoint with a position and a *scale*: how big the feature is.
- **Orientation.** Each keypoint gets a dominant gradient direction, so the description can be rotated to a standard orientation.
- **Description.** The patch around the keypoint, sized by its scale and rotated by its orientation, is split into a 4×4 grid, and each cell gets an 8-bin histogram of gradient directions: a 128-number descriptor. Because it is normalized for scale and rotation, the same point seen from a bit further away or at a slight angle gives a similar descriptor.

3.2 Matching and the ratio test

For each descriptor in image A, the closest descriptor in image B (by Euclidean distance) is a candidate match. But "closest" is not enough: in a repeated pattern, like carpet or a row of identical chairs, many points look almost the same, and the closest one is often wrong.

Lowe's *ratio test* compares the closest match with the second closest. If the closest distance is less than 0.75 times the second-closest distance, the match is distinctive and kept; if the two are similar, the match is ambiguous and dropped. The given `match` function also runs the test in both directions and keeps a match only if each point is the other's choice.

3.3 Writing `features.py`

Task 2: Complete `features.py`:

- **TODO 3.1** `detect_sift`: create a detector with `cv2.SIFT_create` and call `detectAndCompute` (Hint: final shape is row,col,3).
- **TODO 3.2** Still in `detect_sift`: remove lens distortion from the keypoint positions with `cv2.undistortPoints`, using your calibration. Pass `P=K` so the result stays in pixels.
- **TODO 3.3** `ratio_test`: call the matcher's `knnMatch` with $k = 2$ and apply the ratio test.

Suppressing the upper background. Our reconstruction targets are near the bottom of the image. Features on people, chairs, and distant walls can dominate the matches without helping us reconstruct the objects. Moving people can also produce correspondences that are inconsistent with the stationary scene.

In `detect_sift`, immediately after converting the image to grayscale and before running SIFT, add:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray[:280, :] = 0  # Black out rows 0 to 279 before SIFT
```

The first line is already in the starter code; add only the second. For our 640×480 images, this keeps the bottom 200 rows. Check that both objects remain entirely below the cutoff in every image; adjust the cutoff if necessary.

This changes pixel intensities without cropping or resizing, so pixel coordinates and the intrinsic matrix K stay unchanged. The saved match visualizations will still show the original images: only the grayscale image passed to SIFT is blacked out.

Every later step does geometry on the undistorted positions (`pixels`); the original positions (`raw_pixels`) are only for drawing. Then:

```
python3 features.py ../captures/run_<date>_<time>              # matches between neighbors
python3 features.py ../captures/run_<date>_<time> --image 5    # draw image 5's keypoints
python3 features.py ../captures/run_<date>_<time> --pair 5 6   # draw matches between 5 and 6
```

Pictures go in `output/run_<date>_<time>/`. Look at where the keypoints are, and where the matches are. The summary tells you how many matches each neighboring pair has; you will pick a pair from it in the next section.

4 Epipolar Geometry

4.1 The epipolar constraint

Suppose a point $\mathbf{x}_1$ in image 1 matches $\mathbf{x}_2$ in image 2. The 3D point lies somewhere on the ray from camera 1 through $\mathbf{x}_1$. Seen from camera 2, that ray is a line in image 2: the *epipolar line* of $\mathbf{x}_1$. Whatever the point's depth, its match $\mathbf{x}_2$ must lie on that line.

For every pair of cameras there is a 3×3 matrix F, the *fundamental matrix*, that encodes this. With pixels written as homogeneous vectors $\mathbf{x} = (u, v, 1)^\top$:

$$\mathbf{x}_2^\top F \, \mathbf{x}_1 = 0, \qquad \boldsymbol{\ell}_2 = F\mathbf{x}_1 \text{ is the epipolar line of } \mathbf{x}_1 \text{ in image 2}, \qquad \boldsymbol{\ell}_1 = F^\top \mathbf{x}_2.$$

A line $\boldsymbol{\ell} = (a, b, c)$ is the set of pixels with $au + bv + c = 0$. F is defined only up to scale and has rank 2, so it has 7 degrees of freedom.

All epipolar lines in an image pass through one point, the *epipole*: the pixel where the other camera's center appears. If camera 2 is off to the side of camera 1 (the robot moved sideways), the epipole is far off to the side and the lines are nearly parallel.

4.2 How `cv2.findFundamentalMat` works

You will call OpenCV for this step, but you should know what it does.

The 8-point algorithm. Writing out $\mathbf{x}_2^\top F \mathbf{x}_1 = 0$ for one match gives one equation that is linear in the nine entries of F:

$$\begin{bmatrix} u_2u_1 & u_2v_1 & u_2 & v_2u_1 & v_2v_1 & v_2 & u_1 & v_1 & 1 \end{bmatrix} \mathbf{f} = 0,$$

where $\mathbf{f}$ is F stacked into a vector. Stack $n \geq 8$ matches into a matrix A and solve $A\mathbf{f} = 0$: the solution (up to scale) is the right singular vector of A with the smallest singular value. With noise, the resulting F is not exactly rank 2, so the algorithm takes the SVD of F, sets its smallest singular value to zero, and multiplies back together.

Normalization. Pixel coordinates are in the hundreds while the last column of A is 1, so A is badly conditioned and small noise gives large errors. The *normalized* 8-point algorithm (Hartley) first shifts each image's points so their centroid is at the origin and scales them so their average distance from it is $\sqrt{2}$, solves for F in those coordinates, then undoes the transformation.

RANSAC. Some of your matches are wrong, and a single wrong match can ruin a least-squares fit. RANSAC repeatedly picks a random minimal set of matches, fits F to just those, and counts how many of *all* the matches agree with it (their distance from their epipolar line is under a threshold, 1.5 px here). It keeps the F with the most agreement, then refits on all of those inliers. If a fraction w of the matches are correct and each sample uses s matches, you need about $N = \log(1-p)/\log(1-w^s)$ samples to find an all-correct sample with probability p. OpenCV uses the 7-point minimal solver for each sample and the 8-point algorithm to refit.

4.3 Choosing a pair

Pick two neighboring images from the `features.py` summary that have many matches (100 or more), show your objects, and were taken about 10–20 cm apart. Avoid pairs where the robot only turned in place, and pairs that mostly see distant walls: points several meters away barely move between two nearby cameras, so their depth is poorly determined.

4.4 Writing the epipolar part of `two_view.py`

Task 3: Complete Section 4 of `two_view.py`:

- **TODO 4.1** `estimate_fundamental`: call `cv2.findFundamentalMat` with RANSAC.
- **TODO 4.2** `epipolar_lines`: call `cv2.computeCorrespondEpilines`.
- **TODO 4.3** `epipolar_distances`: the distance from each $\mathbf{x}_2$ to the epipolar line of its match.

```
python3 two_view.py ../captures/run_<date>_<time> 5 6
```

It prints how many matches are RANSAC inliers and how far the inliers are from their epipolar lines, and draws a dozen matches and their epipolar lines in `output/.../epipolar_5_6.png`. Each colored dot in one image should sit on the line of the same color in the other image. (Until you finish Section 5, the script stops here.)

Checkpoint 2

Submit a checkoff request at `https://tinyurl.com/ee106a-fa26lab`. At this point you should be able to:

- Show your capture folder, and a keypoint and a match picture from `features.py`.
- Show your epipolar line picture.
- Explain why the carpet has so few matches, even though it has many keypoints
- Explain what $\mathbf{x}_2^\top F \mathbf{x}_1 = 0$ means geometrically. Why does a match have to lie on its epipolar line?
- Point out where the epipole is in your picture, and explain why your epipolar lines are (or aren't) nearly parallel
- Explain why we can't fit F to all the matches with least squares, without RANSAC
- Explain why we black out the upper part of the image before running SIFT. Does a decrease in the total number of matches mean the result is worse? Why can we keep the same intrinsic matrix K?

---

5 Two-View Reconstruction

5.1 From F to the essential matrix

F works in pixels. Once you know K, you can work with rays instead: $\hat{\mathbf{x}} = K^{-1}\mathbf{x}$. The constraint on rays is the *essential matrix* E:

$$\hat{\mathbf{x}}_2^\top E\, \hat{\mathbf{x}}_1 = 0, \qquad E = [\mathbf{t}]_\times R, \qquad F = K^{-\top} E K^{-1} \implies E = K^\top F K,$$

where camera 2's coordinates are $X_2 = RX_1 + \mathbf{t}$ and $[\mathbf{t}]_\times$ is the matrix that computes a cross product with $\mathbf{t}$. A valid E has two equal singular values and one zero singular value.

5.2 Four solutions and the cheirality check

Decomposing E with the SVD gives two possible rotations and a translation whose sign is unknown: four candidate $(R, \mathbf{t})$. Only one puts the scene in front of both cameras. `cv2.recoverPose` tries all four, triangulates the matches for each, and keeps the one with the most points at positive depth in both cameras (the *cheirality* check).

5.3 Triangulation

Each camera has a 3×4 projection matrix $P = K[R \mid \mathbf{t}]$. Put camera 1 at the origin: $P_1 = K[I \mid \mathbf{0}]$ and $P_2 = K[R \mid \mathbf{t}]$. A 3D point X (homogeneous) and its pixel $\mathbf{x} = (u, v)$ satisfy $\mathbf{x} \sim PX$, which gives two linear equations,

$$u\,\mathbf{p}_3^\top X - \mathbf{p}_1^\top X = 0, \qquad v\,\mathbf{p}_3^\top X - \mathbf{p}_2^\top X = 0,$$

where $\mathbf{p}_k^\top$ is row k of P. Two views give four equations in the four unknowns of X; the SVD gives the best solution. `cv2.triangulatePoints` does this for every match.

5.4 Scale, and where SLAM comes in

E only fixes the *direction* of $\mathbf{t}$: shrink the whole scene and the distance between the cameras by the same factor and every image stays identical. `recoverPose` returns $\|\mathbf{t}\| = 1$, so your point cloud is in units of "one baseline". The robot's SLAM poses tell you how far apart the two cameras really were, and that fixes the scale.

To use them you need each camera's pose in the map: the robot's pose from SLAM chained with where the camera sits on the robot,

$$T_{\text{map}\to\text{cam}} = T_{\text{map}\to\text{base_link}}\; T_{\text{base_link}\to\text{cam}}.$$

5.5 Writing the rest of `two_view.py`

Task 4: Complete Section 5 of `two_view.py`:

- **TODO 5.1** `essential_from_fundamental`: $E = K^\top F K$.
- **TODO 5.2** `relative_pose`: call `cv2.recoverPose`.
- **TODO 5.3** `triangulate`: build P_1 and P_2, call `cv2.triangulatePoints`, convert from homogeneous coordinates, and mark the points in front of both cameras.
- **TODO 5.4** `camera_pose_in_map`: chain $T_{\text{map}\to\text{base_link}}$ with `T_BASE_CAM`.
- **TODO 5.5** `metric_scale`: the factor that turns the unit-length $\mathbf{t}$ into the SLAM baseline.

```
python3 two_view.py ../captures/run_<date>_<time> 5 6
python3 view_viser.py ../captures/run_<date>_<time> --two-view
```

The script prints the relative pose it found, followed by the reprojection error of the triangulated points. Open `http://localhost:8080` to see the two cameras (with their pictures) and the points in 3D. Click a camera to look through it.

6 Multi-View Reconstruction

Two images only see part of the objects. `multiview.py` uses all of your images:

1. Match every pair of images, keeping matches that are `findFundamentalMat` inliers *and* lie near the epipolar lines predicted by the SLAM poses.
2. Link matches into *tracks*: if keypoint a in image 1 matches b in image 2, and b matches c in image 5, then a, b and c are one physical point.
3. Triangulate each track from all of its images at once (the same linear equations as before, two per image), using the SLAM camera poses.
4. Keep points that are in front of every camera, reproject within 4 px in every image, and are seen from directions at least 2 degrees apart.

Projecting needs each camera's pose the other way around from `camera_pose_in_map`: $(R, \mathbf{t})$ with $X_{\text{cam}} = RX_{\text{map}} + \mathbf{t}$.

Task 5: Complete `multiview.py`:

- **TODO 6.1** `world_to_camera`: invert the camera's pose in the map.
- **TODO 6.2** `reprojection_errors`: project points with `cv2.projectPoints` and measure the distance to the observed pixels.

```
python3 multiview.py ../captures/run_<date>_<time>
```

This writes `multiview.ply` (open it in MeshLab or CloudCompare if you like) and `multiview.npz`, the input to the next section.

7 Bundle Adjustment

The SLAM poses are good, but not perfect, and each point was triangulated with the poses held fixed. *Bundle adjustment* refines everything together: every camera pose and every 3D point, to minimize the total reprojection error over all images,

$$\min_{\{R_i, \mathbf{t}_i\}, \{X_j\}} \sum_{(i,j)\ \text{observed}} \rho\Big(\big\| \pi(K, R_i, \mathbf{t}_i, X_j) - \mathbf{x}_{ij} \big\|^2 \Big) \ + \ \text{(a penalty for moving far from the SLAM poses)},$$

where π projects a point into a camera and ρ is a robust (Huber) loss that limits the influence of the occasional bad match. It is a large nonlinear least-squares problem, solved here with Levenberg-Marquardt. The penalty keeps the solution in the SLAM map's frame and scale, which the images alone cannot pin down.

This part is given. Run it and look at what it does:

```
python3 bundle_adjust.py ../captures/run_<date>_<time>
python3 view_viser.py ../captures/run_<date>_<time>
```

In the viewer, use the *Iteration* slider or *Play* to step through the optimization. The cameras and points move from their starting positions (grey frustums are the SLAM poses), and the reprojection error drops on the plot. The real corrections are millimeters to a few centimeters, so the *Exaggerate motion* slider magnifies them.

Checkpoint 3

Submit a checkoff request at `https://tinyurl.com/ee106a-fa26lab`. At this point you should be able to:

- Show your two-view reconstruction in the viewer.
- Show your bundle adjustment in the viewer.
- Explain why two images alone can't tell you how big the scene is, and how you got meters
- Explain why points several meters away are less accurate than points on your objects
- Say how far bundle adjustment moved the cameras, and what that tells you about the SLAM poses

---

8 Before you Leave

Save your code (your captures are small enough to commit too):

```
cd ~/ros_workspaces
git add -A lab6
git commit -m "Complete Lab 6"
git push origin main
```

The `.venv` and `output` folders are not committed; you can recreate them.

The lab workstations and the TurtleBots are shared. Before you go:

- Ctrl+C out of every terminal, including the viewer. Do not use Ctrl+Z.
- Ctrl+C the bringup and camera in your ssh sessions and type `exit`.
- Turn the TurtleBot off and plug its battery back into the charging station.
- Return the checkerboard and your objects.
- `exit` your Distrobox container, and log out of your `ee106a-*` account.

Congratulations on finishing Lab 6!